

当 Dense Transformer 继续扩大规模变得越来越困难时，模型架构需要从“所有输入都用全部参数”转向“不同输入只激活部分能力”，也就是 selective architecture。

1. 为什么需要 Selective Architecture?

传统大语言模型，比如 GPT-3，是典型的 dense Transformer。它的特点是：

每一个输入 token，都会经过模型的全部参数。

比如 GPT-3 有 175B 参数，一个输入来了，虽然不是每一步都“显式使用”全部参数，但整体计算路径是密集的。模型越大，训练和推理成本越高，还会遇到并行瓶颈：模型太大，需要拆到很多机器上，机器之间通信会成为瓶颈。

通俗来说：

Dense Transformer 就像一个公司里所有问题都要经过全体专家开会。

模型越大，专家越多，协调成本越高，最后**不是专家不够聪明，而是“开会成本”太高。**

所以这份讲义提出一个问题：

能不能让不同输入只调用一小部分参数？

这就是 selective architecture 的核心思想。

2. 这份讲义主要讲两类选择性架构

讲义重点讨论了两种方向：

1. Mixture of Experts, MoE, 混合专家模型
2. Retrieval-based / Retrieval-augmented models, 检索增强模型

它们共同解决的问题是：

不要让模型把所有知识、所有能力都塞进一个 dense 网络里，而是让模型在需要时选择性调用某些专家或外部知识。

一、Mixture of Experts：让模型只调用少数专家

3. MoE 的基本思想

MoE 的核心是：模型里有很多个 expert，每个 expert 是一个子网络。面对一个输入时，模型先用一个 **gating function** 判断应该调用哪些专家，然后只激活其中少数几个专家。

通俗来说：

Dense Transformer 像“一个超级全能员工”，什么都自己做。

MoE 像“专家委员会”，数学问题找数学专家，历史问题找历史专家，代码问题找代码专家。

每次只请相关专家出手，不让所有专家都参与。

这就带来一个好处：

模型总参数可以很大，但每个输入实际使用的参数很少。

也就是：

参数规模变大，但单次计算成本不一定同比例变大。

4. MoE 的关键机制：Gate 怎么选专家？

讲义中先介绍了普通 MoE：

每个 expert 都有一个权重，gate 会给每个 expert 一个概率。最后输出是所有专家输出的加权平均。

但是这样有一个问题：

如果每个 expert 都要算一遍，那就没有省计算。

所以真正重要的是 **sparse MoE**：

只选 top-1 或 top-2 expert。

比如 gate 原本给出的概率是：

```
[0.04, 0.80, 0.01, 0.15]
```

如果只选 top-2，就变成：

```
[0, 0.84, 0, 0.16]
```

也就是说，只调用第 2 个和第 4 个专家。

通俗来说：

不是所有医生都给你会诊，而是先分诊，然后只找最相关的两个医生。

5. MoE 放在 Transformer 里的位置

一个普通 Transformer block 通常包括两部分：

1. Self-Attention：让 token 之间互相交流。
2. Feed-Forward Network，FFN：对每个 token 独立处理。

MoE 通常不是把整个 Transformer 都换成专家，而是把 **FFN 层替换成 MoE FFN**。

也就是说：

普通 Transformer：

Attention → FFN

MoE Transformer：

Attention → MoE-FFN

为什么这么做？

因为 FFN 本来就是每个 token 独立处理的，非常适合按 token 分配不同 expert。

通俗来说：

Attention 负责“大家开会交流上下文”；

FFN 负责“每个 token 自己加工理解”；

MoE 就是在“个人加工”这一步，引入不同专家。

6. MoE 最大的技术难点：专家负载均衡

MoE 听起来很美好，但有一个大问题：

如果 gate 总是选择同一个 expert，其他 expert 就荒废了。

这会导致两个问题：

1. 计算资源浪费：很多 expert 没被用。
2. 训练失败：没被选中的 expert 没有梯度，无法学习。

所以讲义强调：

MoE 的关键不是有多少专家，而是能不能让专家都被合理使用。

这就需要 load balancing，也就是负载均衡。

通俗来说：

如果公司里所有问题都丢给一个最强专家，那其他专家永远得不到锻炼，整个专家系统就退化成一个普通模型。

为了解决这个问题，MoE 会引入辅助损失函数，鼓励 gate 不要总是选同一个专家。

7. 几种代表性 MoE 模型

7.1 Sparsely-Gated MoE / GShard: Top-2 Experts

这一类方法通常每个 token 选择两个专家。

优点是更稳定，因为不完全依赖一个专家；缺点是计算量比 top-1 大一些。

7.2 Switch Transformer: Top-1 Expert

Switch Transformer 更激进：

每个 token 只选择一个 expert。

这样计算更省，但训练稳定性更难，需要一些技巧，比如：

- FP32 到 FP16 的选择性转换
- 更小的参数初始化
- expert dropout
- expert parallelism

讲义提到 Switch Transformer 训练了 **1.6T 参数模型**，并且相较 T5-XXL 有明显预训练速度提升。

通俗来说：

Switch Transformer 像是极简版 MoE：每个问题只派一个专家处理，效率最高，但分诊系统必须非常可靠。

7.3 BASE Layers：把专家分配变成优化问题

BASE 的思路是：

不是每个 token 独立选专家，而是在整个 batch 上做一个全局分配优化。

目标是：

每个 token 分到合适专家，同时每个专家负载严格均衡。

通俗来说：

普通 MoE 像每个病人自己排队找医生，容易某个医生爆满。

BASE 像医院统一调度挂号，既考虑病人需要，也保证医生工作量均衡。

缺点是分配本身需要额外计算；优点是更稳定。

8. GLaM：MoE 真的能挑战 GPT-3 吗？

讲义介绍了 Google 的 GLaM。

它的特点：

- 1.2T 参数，比 GPT-3 的 175B 大很多。
- 64 个专家。
- 每个 token 只激活约 95B 参数，大约是总参数的 8%。
- 使用 top-2 experts。
- 训练成本约为 GPT-3 的三分之一。
- 在 zero-shot 和 one-shot 上表现强，尤其是知识密集型任务。

这说明 MoE 的核心优势是：

总模型很大，但每次调用很便宜。

通俗来说：

GLaM 像一个拥有 1.2 万名专家的大型咨询公司，但每个项目只调用其中一小组人，所以既有庞大知识储备，又不需要每次全员上阵。

不过讲义也提醒：这些模型和 GPT-3 的直接比较并不完全简单，因为评测设定、数据、few-shot 能力等都有差异。

9. MoE 的更激进方向：去中心化训练

讲义还讨论了 decentralized MoE。

这个想法很有意思：

既然 MoE 的每个 expert 可以相对独立，**能不能让不同 expert 分布在不同普通电脑上？**

比如像 Folding@Home 那样，让大量志愿者贡献算力。

但这里有巨大挑战：

- 普通电脑节点不稳定。
- 家用网络带宽远低于超算集群。
- 节点随时掉线。
- 通信成本很高。

通俗来说：

理想上，可以让全球电脑一起训练一个大模型；
现实中，最大瓶颈不是算力，而是网络通信和系统稳定性。

这部分的意义在于：MoE 不只是提升大公司训练效率，也可能带来更去中心化的模型训练范式。

二、Retrieval-based Models：不要全靠模型记忆，而是学会查资料

10. 检索增强模型的基本思想

第二类 selective architecture 是 retrieval-based models。

它的核心思想是：

模型不需要把所有知识都背在参数里，而是可以在生成前先检索相关资料。

流程是：

输入问题 → 检索相关文档 → 结合文档生成答案

通俗来说：

Dense LM 像闭卷考试，答案都要靠脑子里记住。
Retrieval-augmented model 像开卷考试，可以先查资料，再作答。

例如：

问题：

What is the capital of Canada?

检索到：

Ottawa is the capital city of Canada.

生成答案：

Ottawa

11. RAG: Retrieval-Augmented Generation

RAG 的形式可以理解为两部分：

1. **Retriever**：负责找资料。
2. **Generator**：负责根据资料生成答案。

讲义中提到，RAG 通常使用 Dense Passage Retrieval，DPR，来做检索。

DPR 的思路是：

- 用一个 BERT 编码问题。
- 用另一个 BERT 编码文档。
- 通过向量相似度找到最相关的文档。
- 再把检索到的文档和问题一起交给 BART 生成答案。

通俗来说：

Retriever 像搜索引擎。
Generator 像写答案的人。
RAG 就是“先搜索，再写作”。

这和普通语言模型最大的区别是：

普通语言模型只能依赖参数里的记忆；RAG 可以依赖外部文档库。

12. RAG 和 MoE 的相似性

这份讲义把 MoE 和 RAG 放在一起讲，是因为它们有一个共同点：
都不是对所有资源做 dense computation，而是先选择，再计算。

MoE 是：

输入 → 选择专家 → 调用专家

RAG 是：

输入 → 选择文档 → 基于文档生成

所以两者本质上都是 selective architecture。

区别是：

类型	被选择的对象	选择后做什么
MoE	模型内部专家参数	激活部分参数计算
RAG	外部文档 / 数据库	读取资料后生成答案

通俗地说：

**MoE 是“找内部专家”；
RAG 是“查外部资料”。**

13. RETRO：用更少参数 + 巨大检索库

讲义还介绍了 DeepMind 的 RETRO。

RETRO 的特点是：

- 模型只有 7B 参数，比 GPT-3 少很多。
- 但它有一个 2T tokens 的外部检索库。
- 使用 frozen BERT 做检索。
- 在语言建模和知识问答任务上表现很好。

这说明一个重要趋势：

模型能力不一定只来自参数规模，也可以来自外部记忆库。

通俗来说：

一个 7B 参数模型，如果会查 2T tokens 的资料，可能比一个更大但只能靠记忆模型更有效。

这对实际应用非常重要，因为很多企业知识、金融数据、制度文件、研究报告不适合全部塞进模型参数里，而更适合做成外部知识库。

三、这份讲义的核心判断

14. Dense Transformer 的扩展正在遇到天花板

讲义不是说 Dense Transformer 不行，而是说：

只靠把 dense Transformer 越堆越大，越来越难。

困难包括：

- 训练成本高。
- 网络通信瓶颈。
- 参数规模增长带来的边际收益下降。
- 所有输入都走全部参数，计算效率不高。

15. Selective Architecture 是继续扩展模型能力的重要方向

MoE 和 RAG 都是在回答同一个问题：

如何让模型拥有更大的能力池，但每次只调用必要的一小部分？

这和人类解决问题很像：

- 遇到专业问题，找专家。
- 遇到知识问题，查资料。
- 不需要每次都动用全部大脑资源。

所以 selective architecture 的本质是：

把“模型规模”与“单次计算成本”解耦。

这句话很关键。

Dense Transformer 中，模型越大，单次计算通常越贵。

Selective Architecture 希望做到：

总能力很大
但每次使用很少一部分

四、通俗类比总结

可以用一个公司来理解这份讲义。

Dense Transformer

像一个超级员工，所有事情都自己处理。

优点：

- 简单统一。
- 能力稳定。
- 泛化能力强。

缺点：

- 越做越大，成本越来越高。
- 所有任务都走同一套计算路径，效率低。

MoE

像一个专家组织。

- 数学问题找数学专家。
- 编程问题找编程专家。
- 历史问题找历史专家。

优点：

- 总能力可以很大。
- 每次只调用少数专家，计算更省。

难点：

- 怎么正确分配专家？
- 怎么避免某些专家被过度使用？
- 怎么让训练稳定？

RAG / Retrieval

像一个会查资料的人。

- 不要求把所有知识背下来。
- 先检索，再回答。
- 外部知识可以更新。

优点：

- 适合知识密集型任务。
- 可解释性更好，因为能看到检索来源。
- 知识库可以更新，不必重新训练模型。

难点：

- 检索质量决定答案上限。
- 检索错了，生成也容易错。
- 泛化能力是否能完全替代 dense model 仍不确定。

五、对你理解大模型架构的启发

这份讲义对当前 AI 系统设计很有启发，尤其适合你从“金融业务 + AI 工程”角度理解。

1. 大模型不只是“参数越大越好”

更重要的是：

参数怎么被组织、调用和路由。

MoE 告诉我们，未来模型能力可能不是一个密集整体，而是很多专家模块的组合。

2. 企业级 AI 很可能天然适合 RAG

因为企业知识通常具有这些特点：

- 经常更新。
- 来源分散。
- 需要可追溯。
- 不适合全部写进模型参数。

所以 RAG 对金融、券商、资管、研究报告、制度文件、KPI 规则等场景非常实用。

3. Agent 系统也可以看作一种 selective architecture

从更宽的角度看，Agent 也是 selective 的：

用户问题 → 判断任务类型 → 选择工具 / 检索资料 / 调用模型 / 生成报告

这和 MoE、RAG 的思想是一脉相承的：

不是所有任务都靠一个模型硬答，而是根据任务选择合适资源。

最后一句话总结

这份讲义讲的是：大模型继续扩展，不能只靠把 dense Transformer 做得更大，而要让模型学会“选择性调用资源”——MoE 是选择内部专家，RAG 是选择外部知识。它们共同代表了从“一个大脑全包”走向“专家系统 + 外部记忆”的架构演进。